KOÇ UNIVERSITY

COMP 201 – Spring 2026
Assignment 1 – Strings in C: The Imitation Game

Assigned: 24 February 2026, Due: 10 March 2026 23:59

Lead TA: Deniz Bilge Akkoç (dakkoc25@ku.edu.tr)



1 Introduction

The year is 1943. Britain is at war, and at a secret estate called Bletchley Park, a small team of mathematicians and linguists is working day and night to crack the coded messages of the enemy. You have just been recruited to join this team, and your supervisor is none other than Alan Turing himself.

"Welcome to Bletchley," Turing says, barely looking up from a stack of papers. "Every day, the enemy sends hundreds of encrypted messages. Some use a simple letter-shifting scheme we call the **Caesar cipher**. Others use something far more devious — the **Vigenère cipher**, where each letter in the message is shifted by a different amount determined by a secret keyword."

He slides a folder across the desk. "Inside you'll find several intercepted messages. Some come with intelligence — a word we know appears in the plaintext, or the length of the enemy's keyword. Others come with nothing at all. Your job is to use every tool at your disposal — known words, letter frequencies, common English vocabulary — to break these codes and reveal their contents."

He pauses and adds: "And one more thing. We're low on resources, so everything must be written in C. No dynamic memory allocation. Keep it lean."

You nod, take the folder, and sit down at your desk. The fate of the war effort may depend on what you build next.

Welcome to Bletchley Park — it's time to break some codes!

2 Background: The Ciphers

This section explains the two encryption schemes used throughout the assignment. Make sure you understand both before starting the tasks.

2.1 The Caesar Cipher

The Caesar cipher is one of the oldest and simplest encryption methods, named after Julius Caesar, who reportedly used it for military correspondence. It works by shifting every letter in the plaintext by a fixed number of positions in the alphabet.

Given a **shift value** N (where $1 \leq N \leq 25$):

- Each letter is replaced by the letter N positions later in the alphabet, wrapping around from Z back to A.
- For example, with $N = 3$: A $\rightarrow$ D, B $\rightarrow$ E, …, W $\rightarrow$ Z, X $\rightarrow$ A, Y $\rightarrow$ B, Z $\rightarrow$ C.
- The cipher preserves letter casing: an uppercase letter in the plaintext remains uppercase after encryption, and likewise for lowercase.
- Non-alphabetical characters (spaces, punctuation, digits) are **not** modified.

Mathematically, for a letter with index p (where A $= 0$, B $= 1$, …, Z $= 25$):

$$\text{Encrypted index} = (p + N) \bmod 26$$

$$\text{Decrypted index} = (c - N + 26) \bmod 26$$

Example With shift $N = 5$:

```
Plaintext: ATTACK AT DAWN
Encrypted: FYYFHP FY IFBS
```

2.2 The Vigenère Cipher

The Vigenère cipher is a polyalphabetic substitution cipher, meaning it uses a **different shift for each letter** of the plaintext. The shifts are determined by a repeating **keyword**.

Each letter of the keyword corresponds to a shift value: A $= 0$, B $= 1$, C $= 2$, …, Z $= 25$.

Encryption process:

- Write the plaintext.
- Write the keyword repeatedly underneath it, letter by letter, **but only under alphabetical characters**. Non-alphabetical characters (spaces, punctuation, digits) do not consume a key letter.
- For each plaintext letter, add the corresponding key letter's shift value (modulo 26).
- Letter casing is preserved: the output letter has the same case as the input plaintext letter.
- The key itself is treated as case-insensitive: `ENIGMA` and `enigma` produce the same result.

Decryption is the reverse: subtract the key letter's value instead of adding it.

$$\text{Encrypt: } c_i = (p_i + k_{i \bmod L}) \bmod 26$$
$$\text{Decrypt: } p_i = (c_i - k_{i \bmod L} + 26) \bmod 26$$

where L is the length of the keyword and the index i counts *only* alphabetical characters.

Example With key `KEY`:

```
Plaintext: HELLO WORLD
Key letters: KEYKE YKEYK (spaces don't consume a key letter)
Encrypted: RIJVS UYVJN
```

Walkthrough of first three characters:

- H (7) + K (10) = R (17)
- E (4) + E (4) = I (8)
- L (11) + Y (24) = J (35 mod 26 = 9)

3 Logistics

This is an individual project. All hand-ins are electronic. Clarifications and corrections will be announced on LearnHub.

4 Handout Instructions

1. Accept the GitHub Classroom assignment using the following link:
   https://classroom.github.com/a/ox$_q$R6m

2. Clone the GitHub repository created for you to a Linux machine where you plan to work (Replace `USERNAME` with your GitHub username.):

   ```
   $ git clone https://github.com/COMP201-Spring-2026/assignment-1-strings-in-C-USERNAME.git
   ```

   You MUST make sure your code works on our Linux servers [linuxpool.ku.edu.tr]. Please refer to assignment 0 for details. All grading is done on linuxpool regardless of your local setup.

3. **[IMPORTANT]** After cloning, you are required to **write** the following honor code in a new file called `honor.txt` and **commit & push** it:
   "I hereby declare that I have completed this assignment individually, without support from anyone else."
   You can create and commit it with:

   ```
   $ echo "I hereby declare that I have completed this assignment individually, \
   without support from anyone else." > honor.txt
   $ git add honor.txt && git commit -m "Add honor code" && git push origin main
   ```

5 Files Overview

This assignment includes the following file/s:

- **`main.c`**: the code file you will implement. It must contain all logic for all tasks. A `main` function is provided that handles terminal subcommands; you must not modify it. You are encouraged to write helper functions (as many as needed) to keep your code modular and readable.

Resources on C File Handling:

- Basics of File Handling in C (GeeksforGeeks)
- C Standard Library – fgets() (TutorialsPoint)
- Reading a File Line-by-Line in C (Stack Overflow)

6 Tasks

6.1 Task 1: Known-Plaintext Caesar Attack (15 pts)

The first intercepted message has been encrypted with a Caesar cipher. The shift value is unknown, but intelligence reports confirm that a specific word (e.g., `ATTACK`) appears somewhere in the original plaintext.

Function Signature

```c
int caesar_known_word(const char *encrypted_path,
                      const char *known_word,
                      const char *output_path);
```

- Read the encrypted file from `encrypted_path`.
- The known word is passed as a string argument. It is guaranteed to appear at least once in the original plaintext.
- For each candidate shift value from 1 to 25, decrypt the entire ciphertext and check whether the known word appears in the result. The comparison must be **case-insensitive** (i.e., the known word `ATTACK` should match `Attack`, `attack`, `ATTACK`, etc.).
- Once the correct shift is found, decrypt the full message and write the plaintext to `output_path`.
- Only alphabetical characters (A–Z, a–z) are shifted during encryption/decryption. All other characters remain unchanged. Letter casing is preserved.

```
Encrypted (shift=3): "Dwwdfn dw gdzq!"
Known word: "ATTACK"
Discovered shift: 3
Decrypted output: "Attack at dawn!"
```

6.2 Task 2: Vigenère Encrypt and Decrypt (15 pts)

The Caesar cipher proved too weak, and intelligence has learned that the enemy now uses the Vigenère cipher. Before you can crack their messages, you need to master the Vigenère algorithm by implementing both encryption and decryption.

6.2.1 Part A — Encryption (7.5 pts)

Function Signature

```c
int vigenere_encrypt(const char *plaintext_path,
                     const char *key,
                     const char *output_path);
```

- Read the plaintext file from `plaintext_path`.
- The key is passed as a string argument (e.g., `ENIGMA`).
- Encrypt the text using the Vigenère cipher as described in Section 2.2.
- Write the encrypted result to `output_path`.
- Non-alphabetical characters pass through unchanged and do not consume a key letter.
- Letter casing is preserved: the encrypted letter has the same case as the original plaintext letter.
- The key is case-insensitive.

6.2.2 Part B — Decryption (7.5 pts)

Function Signature

```c
int vigenere_decrypt(const char *encrypted_path,
                     const char *key,
                     const char *output_path);
```

- Read the encrypted file from `encrypted_path`.
- The key is passed as a string argument.
- Decrypt the text by reversing the Vigenère cipher (subtract key values instead of adding).
- Write the decrypted result to `output_path`.
- All the same rules about casing, non-alphabetical characters, and key handling apply.

6.3 Task 3: Frequency-Based Caesar Cracking (30 pts)

A new intercepted message has arrived, but this time intelligence has no known words to offer. The message is encrypted with a Caesar cipher, and the shift is unknown. You must determine the shift using **letter frequency analysis**.

The idea behind frequency analysis is that in any sufficiently long English text, certain letters appear much more often than others. The letter `E` is the most common (about 12.7% of all letters), followed by `T`, `A`, `O`, and so on (Letter frequency wiki page). If we try all 26 possible shifts and compare the resulting letter distribution to the expected English distribution, the shift that produces the closest match is most likely correct.

Function Signature

```c
int caesar_frequency(const char *encrypted_path,
                     const char *output_path,
                     const char *results_path);
```

- Read the encrypted file from `encrypted_path`.
- Count the occurrences of each letter (A–Z) in the ciphertext. Uppercase and lowercase letters are treated as the same letter for counting purposes.
- Use the following standard English letter frequency table (values are percentages):

```
E: 12.70 T: 9.06 A: 8.17 O: 7.51 I: 6.97 N: 6.75
S: 6.33 H: 6.09 R: 5.99 D: 4.25 L: 4.03 C: 2.78
U: 2.76 M: 2.41 W: 2.36 F: 2.23 G: 2.02 Y: 1.97
P: 1.93 B: 1.29 V: 0.98 K: 0.77 J: 0.15 X: 0.15
Q: 0.10 Z: 0.07
```

- For each candidate shift $s \in \{0, 1, 2, \ldots, 25\}$, decrypt the text with that shift and compute the **chi-squared** (χ^2) statistic comparing the observed letter distribution to the expected English distribution:

$$\chi^2(s) = \sum_{i=\texttt{A}}^{\texttt{Z}} \frac{(O_i - E_i)^2}{E_i}$$

  where O_i is the observed count of letter i in the candidate decryption and E_i is the expected count for letter i, calculated as:

$$E_i = \frac{\text{frequency percentage of letter } i}{100} \times \text{total number of letters in the text}$$

- The shift with the **lowest** chi-squared value is the most likely correct shift, because a lower value means the observed distribution is closer to what is expected for English text.
- Write the chi-squared score for **every** shift (0–25) to `results_path`, marking the best one. Use `float` or `double` for calculations and write scores with 2 decimal places. The results file must follow this exact format:

```
Shift 0: chi2 = 345.21
Shift 1: chi2 = 298.44
...
Shift 7: chi2 = 12.35 <-- best
...
Shift 25: chi2 = 410.88

Best shift: 7 (chi2 = 12.35)
```

- Write the decrypted message (using the best shift) to `output_path`.

6.4 Task 4: Vigenère Key Recovery via Frequency Analysis (40 pts)

An allied listening station has intercepted a lengthy Vigenère-encrypted transmission. Cryptanalysts in the signals division have already determined that the key is exactly K letters long, but the key itself remains unknown. It falls to you to recover it.

The core insight is this: if the key has length K, then every K-th letter in the ciphertext was encrypted with the **same** shift. This means you can split the ciphertext into K separate groups, and each group is effectively a Caesar cipher on its own. By applying frequency analysis to each group independently, you can recover each character of the key.

Function Signature

```
int vigenere_frequency(const char *encrypted_path,
                       int key_length,
                       const char *output_path,
                       const char *results_path);
```

- Read the encrypted file from `encrypted_path`. The key length is passed as an integer argument.
- Extract only the alphabetical characters from the ciphertext (skip spaces, punctuation, digits, etc.) while preserving their order.
- Distribute these characters into K groups based on their position:
  - Group 0 contains characters at positions $0, K, 2K, 3K, \ldots$
  - Group 1 contains characters at positions $1, K{+}1, 2K{+}1, 3K{+}1, \ldots$
  - …and so on up to Group $K{-}1$.
- Apply the chi-squared frequency analysis method from Task 3 to each group independently. For each group, try all 26 possible shifts and select the one with the lowest χ^2 value.
- The optimal shift for group i gives the i-th character of the key: shift 0 = `A`, shift 1 = `B`, …, shift 25 = `Z`.
- Once the full key is recovered, decrypt the entire message using Vigenère decryption (the same logic as Task 2 Part B).
- The message will be long enough that frequency analysis on each column is reliable (at least 200 alphabetical characters per column).
- Write the recovered key information to `results_path` in the following format:

```
Key length: 6
Recovered key: ENIGMA
```

- Write the decrypted text to `output_path`.

6.5 Task 5 (BONUS): Caesar Cracking via Common Word Scoring (20 pts)

For the final challenge, a different approach: instead of analyzing individual letter frequencies, you will score candidate decryptions based on how many **common English words** they contain. The idea is that a correctly decrypted message will contain many frequently used words like “the”, “and”, “to”, and “of”, while an incorrectly decrypted message will contain mostly gibberish.

Function Signature

```
int caesar_word_score(const char *encrypted_path,
                      const char *wordlist_path,
                      const char *output_path,
                      const char *results_path);
```

- Read the encrypted file from `encrypted_path`.
- Read the word list from `wordlist_path`. This file contains common English words, one per line, each followed by an integer score. Higher scores indicate more common words. Example format:

```
the 100
of 80
and 75
to 70
a 65
in 60
is 55
it 50
for 45
that 40
was 35
on 30
are 28
with 26
...
```

  The file may contain up to 50 words.

- For each candidate shift $s \in \{0, 1, 2, \ldots, 25\}$:
  - Decrypt the entire ciphertext using shift s.
  - Split the decrypted text into **words**. A word is defined as a maximal contiguous sequence of alphabetical characters, delimited by any non-alphabetical character (spaces, punctuation, digits, newlines, etc.).
  - For each word in the decrypted text, check (case-insensitively) whether it appears in the common word list. If it does, add that word's score to the total score for shift s.
  - Each occurrence of a word counts separately: if "the" appears 5 times and has a score of 100, it contributes $5 \times 100 = 500$ to the total.
  - Matching must be on **whole words only**. The word "the" appearing inside "theater" does not count — "theater" is a single word and would need to match as "theater" in the word list to score.
- The shift with the **highest** total score is the most likely correct shift.
- Write the score for every shift (0–25) to `results_path`, marking the best one. The results file must follow this exact format:

```
Shift 0: score = 0
Shift 1: score = 45
```

```
...
Shift 13: score = 2750 <-- best
...
Shift 25: score = 30

Best shift: 13 (score = 2750)
```

- Write the decrypted message (using the best shift) to `output_path`.

7 Commands

We will run your program with these subcommands. Filenames can change — do not hardcode them; take them as arguments. Arguments are already handled in the provided `main` function.

```
# Build
$ gcc main.c -o enigma

# Task 1: Caesar decrypt with known word
./enigma caesar-known message1_encrypted.txt ATTACK message1_decrypted.txt

# Task 2a: Vigenere encrypt
./enigma vigenere-encrypt message2_plaintext.txt ENIGMA message2_encrypted.txt

# Task 2b: Vigenere decrypt
./enigma vigenere-decrypt message3_encrypted.txt ENIGMA message3_decrypted.txt

# Task 3: Caesar frequency analysis
./enigma caesar-freq message4_encrypted.txt message4_decrypted.txt message4_results.txt

# Task 4: Vigenere key recovery
./enigma vigenere-freq message5_encrypted.txt 6 message5_decrypted.txt message5_results.txt

# Task 5: Caesar word scoring
./enigma caesar-words message6_encrypted.txt common_words.txt message6_decrypted.txt
    message6_results.txt
```

8 No Dynamic Memory Allocation

- All arrays must be statically allocated. Here are suggested values; you may adjust them as needed:

```c
#define MAX_TEXT_SIZE 100000
#define MAX_LINE_LEN 2048
#define MAX_WORD_LEN 64
#define MAX_WORD_COUNT 50
#define ALPHABET_SIZE 26

// example usage
char text[MAX_TEXT_SIZE];
char decrypted[MAX_TEXT_SIZE];
double english_freq[ALPHABET_SIZE];
```

- Use standard C string functions (`strcpy`, `strcat`, `strcmp`, `strlen`, `strchr`, `tolower`, `toupper`, etc.) to manipulate data.

9 Grading Rubric

Your score will be computed out of a maximum of 100 points based on the following distribution:

Task	Description	Points
Task 1	Caesar Decrypt with Known Word	15
Task 2	Vigenère Encrypt & Decrypt (7.5 + 7.5)	15
Task 3	Frequency-Based Caesar Cracking	30
Task 4	Vigenère Key Recovery via Frequency	40
Task 5	Caesar Cracking via Word Scoring	20

Effective Use of Version Control. You are required to push your changes to the repository frequently. Points will be deducted for pushing only the final version. For each task, there should be at least one commit (with a proper commit message) that includes modifications for that task.

Coding Style. You are expected to write code that is neat and easy to understand. This includes writing modular functions for each task and subtask. Points will be deducted for messy or unreadable code.

10 Academic Integrity

All work on assignments must be done individually unless stated otherwise. You are encouraged to discuss with your classmates about the given assignments, but these discussions should be carried out abstractly. That is, discussions related to a particular solution to a specific problem (either in actual code or in the pseudocode) will not be tolerated. In short, turning in someone else's work, in whole or in part, as your own will be considered a violation of academic integrity. Please note that the former condition also holds for the material found on the web as everything on the web has been written by someone else. See Koç University – Student Code of Conduct.

11 Late Submission Policy

You may use up to 7 grace days (in total) over the semester for the assignments. That is you can submit your solutions without any penalty if you have free grace days left. Any additional unapproved late submission will be punished (1 day late: 20% off, 2 days late: 40% off) and **no submission after 2 days (48 hours) will be accepted.**

IMPORTANT! Grace days are counted in the hard deadline. Therefore, you cannot upload your assignment after 2 days under any circumstances.